

TASK 11 · WEEK 11: FULL STACK APPLICATION DEVELOPMENT

Inkwell

Full Stack Blogging Platform — Project Documentation

Stack: React · Node.js / Express · MongoDB

Repository: see accompanying GitHub submission

Prepared: September 2026

Table of Contents

1. Project Overview & Objectives

- Tech stack

- Project structure

2. Setup & Installation Instructions

- Prerequisites

- Step 1 — Clone the repository

- Step 2 — Backend setup

- Step 3 — Frontend setup

- Step 4 — Running with Docker (optional)

- Environment variables

3. How the Technical Requirements Were Met

4. Full Stack Architecture

5. Authentication Flow

- Registration / login

- Silent refresh

- Logout & password changes

- Protected routes

6. Frontend ↔ Backend Communication

7. State Management

8. Deployment Strategy

9. Testing Evidence

10. Screenshots

1. Project Overview & Objectives

Inkwell is a complete blogging platform built to satisfy the Week 11 full stack assignment. Readers can browse and search published posts; registered users can write, edit and delete their own posts, hold threaded discussions in the comments, and like posts.

The goals of the project were to demonstrate a production-shaped full stack application:

- A React single-page application that talks to a completely separate Node.js/Express REST API
- Real JWT authentication, including short-lived **access tokens** and longer-lived **refresh tokens**
- Protected routes enforced on both the frontend (route guards) and the backend (middleware)
- Full CRUD for blog posts, written through a lightweight built-in rich text editor
- A nested / threaded comment system
- User profiles with editable settings and password changes
- Automated backend tests (Jest + Supertest)

Tech stack

Layer	Technology
Frontend	React 18, React Router 6, Axios, DOMPurify
Backend	Node.js, Express, JWT (<code>jsonwebtoken</code>), bcrypt
Database	MongoDB with Mongoose
Testing	Jest, Supertest, mongodb-memory-server
Deployment	Docker / docker-compose (optional)

Project structure

```

blogging-platform/
├── backend/
│   ├── src/
│   │   ├── config/db.js
│   │   ├── models/
│   │   ├── middleware/
│   │   ├── controllers/
│   │   ├── routes/
│   │   ├── utils/generateTokens.js
│   │   └── server.js
│   ├── tests/
│   ├── package.json
│   └── .env.example
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── context/AuthContext.js
│   │   ├── services/api.js
│   │   └── App.js
│   ├── package.json
│   └── .env.example
├── docker-compose.yml
└── README.md

```

Folder / file	Purpose
backend/src/config/db.js	MongoDB connection
backend/src/models/	User, Post, Comment (Mongoose schemas)
backend/src/middleware/	Auth guard + centralized error handler
backend/src/controllers/	Request handlers / business logic
backend/src/routes/	Express routers
backend/src/utils/generateTokens.js	JWT signing helpers
backend/src/server.js	App entry point
backend/tests/	Jest + Supertest test suite
frontend/src/components/	Navbar, PostCard, CommentList, RichTextEditor, etc.
frontend/src/pages/	Home, Login, Register, PostDetail, CreatePost, etc.
frontend/src/context/AuthContext.js	Global auth state
frontend/src/services/api.js	Axios instance + automatic token refresh
frontend/src/App.js	Route definitions

2. Setup & Installation Instructions

Prerequisites

- Node.js 18+ and npm
- A MongoDB instance — either:
 - MongoDB Community Server running locally, **or**
 - A free MongoDB Atlas cluster

Step 1 — Clone the repository

```
git clone <your-repo-url>
cd blogging-platform
```

Step 2 — Backend setup

```
cd backend
npm install
cp .env.example .env
# edit .env and set MONGO_URI, JWT_ACCESS_SECRET,
JWT_REFRESH_SECRET
npm run dev          # starts the API on http://localhost:5000 with
nodemon
```

Run the automated test suite:

```
npm test              # Jest + Supertest, spins up an in-memory
MongoDB
```

The first test run downloads a small MongoDB binary for `mongodb-memory-server` — it needs an internet connection the first time, then caches it locally.

Step 3 — Frontend setup

In a second terminal:

```
cd frontend
npm install
cp .env.example .env
# edit .env if your API isn't on http://localhost:5000/api
npm start             # starts the React dev server on
http://localhost:3000
```

Open **http://localhost:3000** — register an account and start writing.

Step 4 — Running with Docker (optional)

From the repository root:

```
docker compose up --build
```

This starts MongoDB, the API (port 5000), and the built frontend served by nginx (port 3000).

Environment variables

backend/.env

Variable	Description
<code>PORT</code>	Port the API listens on (default <code>5000</code>)
<code>MONGO_URI</code>	MongoDB connection string
<code>JWT_ACCESS_SECRET</code>	Secret used to sign short-lived access tokens
<code>JWT_REFRESH_SECRET</code>	Secret used to sign refresh tokens (must differ from the access secret)
<code>JWT_ACCESS_EXPIRES_IN</code>	Access token lifetime, e.g. <code>15m</code>
<code>JWT_REFRESH_EXPIRES_IN</code>	Refresh token lifetime, e.g. <code>7d</code>
<code>CLIENT_URL</code>	Origin of the frontend, for CORS + cookies

frontend/.env

Variable	Description
<code>REACT_APP_API_URL</code>	Base URL of the backend API, e.g. <code>http://localhost:5000/api</code>

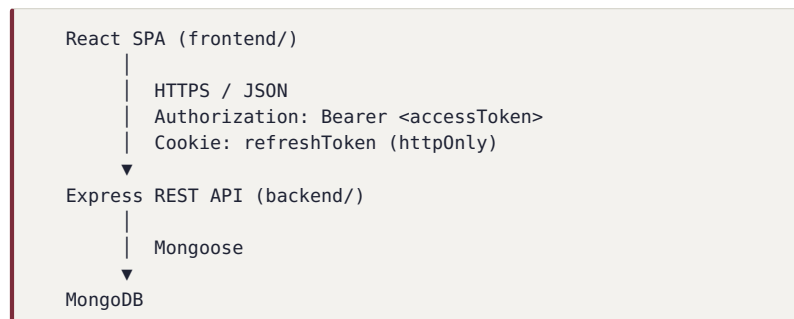
3. How the Technical Requirements Were Met

Requirement	Where it lives
Connect React frontend to Node.js API	<code>frontend/src/services/api.js</code> (axios instance) calling the Express routes in <code>backend/src/routes/</code>
JWT authentication with refresh tokens	<code>backend/src/controllers/authController.js</code> + <code>backend/src/utls/generateTokens.js</code> — see Section 5
Protected routes for authenticated users	Backend: <code>requireAuth</code> middleware. Frontend: <code><ProtectedRoute></code> component
Blog post creation, editing, deletion	<code>backend/src/controllers/postController.js</code> , <code>frontend/src/pages/CreatePost.js</code> / <code>EditPost.js</code>
Comment system with nested comments	<code>backend/src/models/Comment.js</code> (<code>parentComment</code> self-reference) + <code>commentController.js</code> ; rendered recursively in <code>CommentList.js</code>
User profile and settings	<code>authController.js</code> (<code>updateMe</code> , <code>changePassword</code>) + <code>frontend/src/pages/Profile.js</code>

Bonus features beyond the minimum requirements: post search (MongoDB text index), pagination, likes, drafts (`published` flag), and public author profile pages.

4. Full Stack Architecture

The application is split into two independently deployable pieces that only talk to each other over HTTP:



- **Frontend (frontend/)** — a Create React App project. React Router handles client-side navigation; there is no server-rendered HTML beyond the single `index.html` shell.
- **Backend (backend/)** — a stateless Express API under `/api`. It never renders HTML; every response is JSON.
- **Database (MongoDB)** — three collections: `users`, `posts`, `comments`, modeled with Mongoose schemas in `backend/src/models/`.

Request flow for a typical page (viewing a post):

1. React Router matches `/posts/:slug` and renders the `PostDetail` page.
2. `PostDetail` calls `GET /api/posts/:slug` and `GET /api/posts/:id/comments` through the shared `api` axios instance.
3. Express routes the request through `postRoutes.js` into `postController.getPost`, which queries MongoDB via Mongoose, increments the view counter, and returns JSON.
4. React renders the response. The post body — stored as sanitized HTML produced by the rich text editor — is rendered with `dangerouslySetInnerHTML` after a *second* client-side sanitization pass with DOMPurify, as defense in depth against stored XSS.

Why a self-contained rich text editor instead of a library: the post editor is a small `contentEditable` component using `document.execCommand`, rather than a package like Quill or TipTap. This keeps the dependency surface minimal for a course project. Content is sanitized twice — once server-side with `sanitize-html` before it is stored, and again client-side with DOMPurify before it is rendered — so it remains safe despite the simple editing surface.

5. Authentication Flow

The app uses the standard **access token + refresh token** pattern rather than a single long-lived token.

Token	Lifetime	Where it lives	Purpose
Access token	15 minutes	JS memory + <code>localStorage</code> , sent as <code>Authorization: Bearer <token></code>	Authorizes individual API requests
Refresh token	7 days	httpOnly cookie , scoped to <code>/api/auth</code>	Used only to mint a new access token; never touched by client-side JS

Putting the refresh token in an httpOnly cookie — rather than `localStorage`, like the access token — means that even if an attacker found a way to run JavaScript on the page (XSS), they could steal the short-lived access token but **not** the refresh token, limiting the blast radius of a compromise.

Registration / login

- `POST /api/auth/register` or `/login` validates credentials, hashes the password with bcrypt (on register) or compares against the stored hash (on login).
- On success, the server signs an **access token** and returns it in the JSON body, and signs a **refresh token** and sets it as an httpOnly, `SameSite` cookie via `res.cookie('refreshToken', ...)`.
- The frontend stores the access token and the user object in `AuthContext`.

Silent refresh

`frontend/src/services/api.js` installs an axios response interceptor:

- Any request that fails with `401` (expired/invalid access token) triggers a call to `POST /api/auth/refresh-token`.
- The browser automatically attaches the httpOnly cookie to that request — this is why `withCredentials: true` is set on the axios instance, and why CORS is configured with `credentials: true` and a single explicit origin on the backend (wildcard origins cannot be combined with credentialed requests).
- `authController.refreshToken` verifies the refresh token, checks its embedded `tokenVersion` against the user's current `tokenVersion`, and — if valid — issues a **new** access token *and* rotates the refresh token, guarding against replay if a refresh token is ever intercepted.
- The interceptor stores the new access token and retries the original request transparently — the user never sees the failure.

5. If the refresh call itself fails, the interceptor clears local state and dispatches an `auth:logout` event that `AuthContext` listens for, logging the user out in the UI.

This also means a hard page refresh does not log the user out: on mount, `AuthContext` calls `/api/auth/refresh-token` if there is no access token in memory, using the still-valid cookie to re-establish a session silently.

Logout & password changes

- `POST /api/auth/logout` clears the refresh-token cookie.
- Changing a password, or explicitly logging out of *all* devices, increments the user's `tokenVersion` field in MongoDB. Because every refresh token embeds the `tokenVersion` it was issued with, this instantly invalidates every outstanding refresh token without a server-side token blacklist.

Protected routes

- **Backend:** `requireAuth` middleware verifies the access token and attaches `req.user`; routes such as `POST /api/posts` list it before the controller.
- **Frontend:** `<ProtectedRoute>` checks `AuthContext`'s `isAuthenticated` flag and redirects unauthenticated users to `/login`, remembering the page they were headed to.

6. Frontend ↔ Backend Communication

- All communication happens over a single axios instance with `baseUrl` pointed at `REACT_APP_API_URL`.
- Every request and response is JSON except the refresh-token cookie, which travels as a normal `Set-Cookie` / `Cookie` header.
- CORS on the backend is locked to a single configured origin (`CLIENT_URL`) with `credentials: true`, required for the browser to send the httpOnly cookie cross-origin during local development.
- Errors are normalized: the backend's centralized error handler always responds with `{ error: "message" }`, so the frontend can read `err.response?.data?.error` in one consistent place.

7. State Management

The app deliberately avoids a global state library (Redux, Zustand, etc.) — the state surface is small enough that React’s built-in tools are sufficient and easier to follow:

- **Auth / current user** — `AuthContext` (React Context + `useState`), because it is needed across the tree (navbar, protected routes, profile settings) and changes relatively rarely.
- **Page-local data** (a list of posts, a single post and its comments, a profile) — plain `useState` / `useEffect` inside each page component, fetched fresh on mount. There is no cross-page cache; navigating back to a page re-fetches. This is a deliberate simplicity trade-off — a production app with heavier data-sharing needs would reach for React Query or SWR here.
- **Access token** — lives outside React entirely, in a module-level variable inside `services/api.js` (mirrored to `localStorage` so it survives a refresh). It does not need to be reactive state because nothing renders based on its raw value — components only care about the derived `isAuthenticated` boolean from `AuthContext`.

8. Deployment Strategy

The two halves of the app deploy independently.

Database — MongoDB Atlas free tier. Managed, backed up, and reachable from any host without running a dedicated database server.

Backend — any Node-friendly host works, since it is a standard Express app with no special build step:

- *Render / Railway / Fly.io*: point at `backend/`, build command `npm install`, start command `npm start`, and set the environment variables from Section 2 (in particular `MONGO_URI` pointed at the Atlas cluster, and `CLIENT_URL` pointed at wherever the frontend ends up).
- *Docker*: `backend/Dockerfile` builds a minimal `node:20-alpine` image; the root `docker-compose.yml` wires it together with MongoDB and the frontend for a one-command deployment.

Frontend — a static build (`npm run build` produces `frontend/build/`), so it can go on any static host:

- *Vercel / Netlify*: point at `frontend/`, build command `npm run build`, output directory `build`, and set `REACT_APP_API_URL` to the deployed backend's URL.
- *Docker*: `frontend/Dockerfile` is a two-stage build that compiles the app and serves the static files with nginx.

Production considerations actually implemented in the code:

- The refresh-token cookie options switch to `secure: true, sameSite: 'none'` automatically when `NODE_ENV=production`, since a cross-site cookie over HTTPS requires those flags.
- Rate limiting (`express-rate-limit`) is applied to the login/register endpoints to slow down credential-stuffing attempts.
- All user-authored HTML content is sanitized on both write (`sanitize-html`, server) and read (DOMPurify, client).

9. Testing Evidence

The backend has an automated Jest + Supertest suite in `backend/tests/`, run against an **in-memory MongoDB instance** (via `mongodb-memory-server`), so it does not touch a real database and can run in CI:

```
backend/tests/  
├─ setup.js  
├─ auth.test.js  
└─ posts.test.js
```

- `setup.js` — spins up and tears down the in-memory MongoDB for each test file
- `auth.test.js` — registration, login success/failure, protected `/me` route, refresh-token flow
- `posts.test.js` — post CRUD with author-only permissions, nested comment creation and cascading delete, like/unlike toggling

Run it with:

```
cd backend  
npm test
```

How this was verified while building the project: every backend source file was syntax-checked (`node -c`), all dependencies installed cleanly, and the Express app was loaded standalone to confirm every route registers correctly (18 routes across `/api/auth`, `/api/posts`, and `/api/users`). The frontend was verified by running a full production build (`npm run build`) with ESLint warnings treated as errors — it compiles cleanly with zero errors or warnings, which catches import mistakes, unused variables, and most typos across all React source files.

Add a screenshot of a `npm test` run (or your CI pipeline output) here before final submission.

10. Screenshots

(Insert screenshots of the running application here. Suggested set, in order:)

1. Registration page and a successful sign-up
2. Login page
3. Home page with a list of posts, and the search bar in use
4. The rich text editor while writing a post
5. A published post's detail page, including tags and the like button
6. The comment section with at least one nested / threaded reply
7. The profile / settings page
8. A terminal screenshot of `npm test` passing in `backend/`
9. The browser Network tab showing the `Authorization: Bearer ...` header on a request, and the `refreshToken` cookie on the response of `/api/auth/login`